THESIS ARCHITECTURE PROPOSAL — VERSION 2

Predictive Cloud-Native Mobile Edge

Optimized Task Offloading for Smart City Vehicular IoT Networks

Application Domain: Autonomous & Connected Vehicles — Istanbul Urban Network

A fully proactive 3-tier IoT–Fog–Cloud system orchestrating DAG object-detection pipelines for smart vehicles via TOF-MMDE-NSGA-II offline optimisation, dual DQN agents, SDN-managed networking, and GPS-driven proactive handoff.

1. Application Scenario — Smart Cars in Istanbul

The thesis grounds every architectural decision in one concrete, real-world application: a fleet of connected vehicles navigating Istanbul's urban road network. Istanbul is an ideal choice because it combines high vehicle density (the city has one of the highest cars-per-km² ratios in Europe), a complex multi-district topology that naturally maps to a multi-fog-zone architecture, and existing open-access GPS trace datasets from the IETT public transport system that can be used for simulation mobility traces.

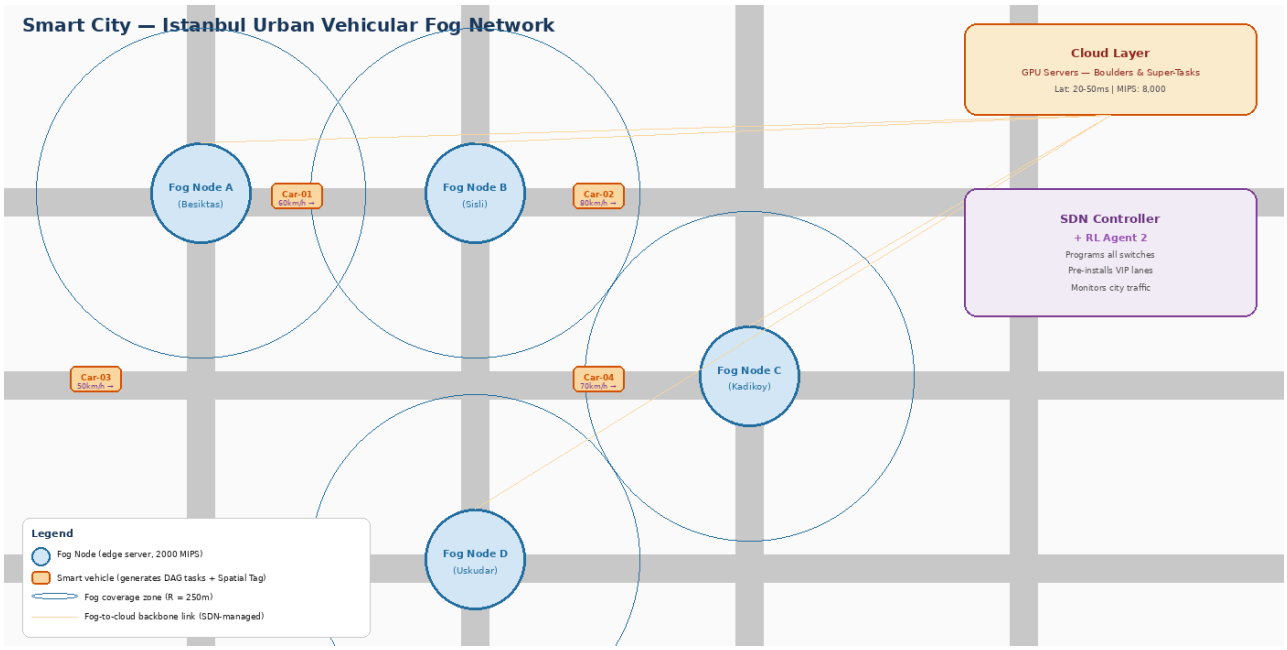

Figure 1: Smart city scenario. Four fog nodes placed at key Istanbul districts (Besiktas, Sisli, Kadikoy, Uskudar). Vehicles generate DAG tasks as they move through overlapping 250m coverage zones. The SDN Controller and RL Agent 2 manage the backbone network. The Cloud handles GPU-intensive boulders.

1.1 The Real Problem Being Solved

Every 100ms, a smart vehicle's camera captures a frame. The vehicle needs to know whether there is a pedestrian, cyclist, or obstacle ahead within 200ms total — fast enough to engage braking systems. The vehicle's onboard computer cannot run a full object detection neural network at this frame rate. It must offload.

The challenge is that the vehicle is moving at 70 km/h — it covers 19.4 metres per second. It may be approaching the boundary of a fog node's coverage zone. If the task is assigned to the wrong fog node, the result may never reach the vehicle — or arrive too late to matter. This is the problem the system solves.

SCENARIO PARAMETERS	City: Istanbul | Vehicles: 50–100 smart cars | Speed: 30–100 km/h | Task: Object detection (10fps camera) | Total deadline: 200ms | Fog nodes: 4 districts | Coverage radius: 250m | Update rate: 10 DAG tasks/second/vehicle

1.2 The DAG Task — Vehicular Object Detection Pipeline

Each camera frame is processed as a 5-step Directed Acyclic Graph. The steps are sequential — each must finish before the next begins, and output data flows from one step to the next. This is what makes it a DAG rather than a simple monolithic task.

DAG Step	Description and cost	EC & routing decision
Step 1: Capture & compress	Runs ON DEVICE. Raw 8MB frame → JPEG 200KB. MI=20, output=200KB.	Device only — too cheap to offload, reduces transmission cost.
Step 2: Pre-process	Resize to 416×416px, normalise pixels. MI=200, output=50KB.	EC = 200÷2000 = 0.10 → Pebble → Fog candidate.
Step 3: Feature extraction	First convolutional layers of YOLO model. MI=2000, output=30KB.	EC = 2000÷2000 = 1.00 → Borderline pebble → Fog (with care).
Step 4: Object classification	Final detection layers — GPU-heavy. MI=8000, output=5KB.	EC = 8000÷2000 = 4.00 → BOULDER → Cloud immediately.
Step 5: Alert generation	Parse boxes, trigger brake/steer alerts. MI=50, output=1KB.	EC = 50÷2000 = 0.025 → Pebble → Fog or device.

KEY INSIGHT	The DAG naturally contains both a boulder (Step 4) and pebbles (Steps 2,3,5). The TOF-Broker handles this split automatically per step. NSGA-II only optimises pebble placement. This is partial offloading — the system runs different steps in different places simultaneously.

2. System Architecture — Full Three-Tier Design

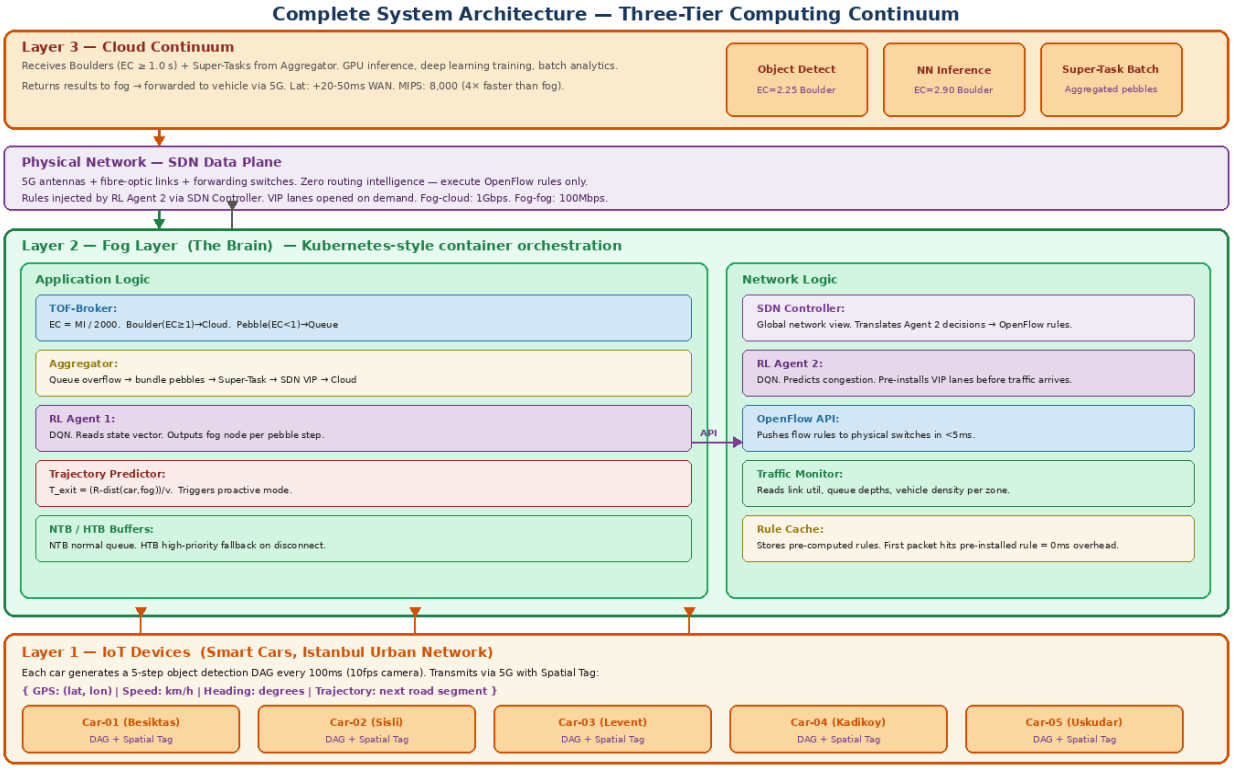

Figure 2: Complete system architecture. Layer 1 (IoT vehicles) generates DAG tasks with Spatial Tags. The SDN Data Plane is the physical network — dumb switches only. Layer 2 (Fog) is split into Application Logic (TOF-Broker, Aggregator, RL Agent 1, Trajectory Predictor, NTB/HTB) and Network Logic (SDN Controller, RL Agent 2, OpenFlow API, Traffic Monitor). Layer 3 (Cloud) handles boulders and Super-Tasks.

2.1 Layer 1 — IoT Vehicles (Smart Cars)

Each smart vehicle runs a lightweight edge daemon that executes Step 1 locally (frame capture and compression) and packages the result as a DAG instance. The daemon then appends the Spatial Tag — a small telemetry header containing the vehicle's current state — and transmits the full packet to the nearest fog node via 5G in approximately 1.5ms.

SPATIAL TAG FORMAT	{ vehicle_id: 'Car-01', lat: 41.0428, lon: 29.0066, speed: 70, heading: 090, trajectory: 'E-5 eastbound toward Kadikoy', timestamp: 1743000000.123 }

The Spatial Tag is what enables everything proactive in this system. Without it, the fog layer can only react to disconnections after they happen. With it, the system can calculate exactly when a vehicle will leave any given coverage zone and act before that happens.

2.2 Physical Network — SDN Data Plane

The physical network is deliberately dumb. Every router, antenna, and switch holds no routing tables and makes no independent decisions. They simply execute flow rules injected by the SDN Controller in Layer 2. This separation — called the control/data plane split — means RL Agent 2 can reprogram the entire city's network routing from a single API call, in under 5ms, without touching any physical hardware.

2.3 Layer 2 — Fog Layer (Application Logic)

Component	Function
TOF-Broker	First component to process every incoming task. Computes EC = task_MI / 2000 per DAG step. Step 4 (EC=4.0) → routed to Cloud immediately, no further processing. Steps 2,3,5 (EC < 1.0) → added to pebble queue for RL Agent 1.
Aggregator	Monitors pebble queue depth. When queue exceeds Q_max (avalanche risk), bundles all queued pebbles into a single compressed Super-Task and routes to Cloud via SDN VIP lane. Prevents cascade failure under peak city traffic loads.
RL Agent 1	Deep Q-Network making real-time placement decisions per pebble step. Pre-trained on NSGA-II Pareto solutions (behavioral cloning). Observes 13-dimensional state vector. Outputs which fog node handles each step. Full architecture in Section 4.
Trajectory Predictor	Reads Spatial Tag. Computes T_exit = (R_fog - dist(car, fog)) / v_closing. Compares with T_exec estimate. Triggers proactive container pre-spin on destination fog node if T_exec > T_exit.
NTB / HTB Buffers	New Task Buffer = normal arrival queue. Handoff Task Buffer = high-priority queue for tasks whose vehicle disconnected unexpectedly mid-execution. HTB tasks preempt NTB tasks and receive maximum CPU until complete. Result held by vehicle ID and delivered on reconnection.

2.4 Layer 2 — Fog Layer (Network Logic)

Component	Function
SDN Controller	Maintains a global view of the physical network: all link utilisations, queue depths, active flows. Receives routing decisions from RL Agent 2 via API. Translates decisions into OpenFlow flow rules and pushes them to physical switches in <5ms.
RL Agent 2	Deep Q-Network embedded in the SDN control plane. Observes 15-dimensional network state vector. Predicts upcoming traffic bursts from learned patterns. Pre-installs OpenFlow flow rules BEFORE traffic arrives. Full architecture in Section 5.
OpenFlow API	Standard communication protocol between the SDN Controller and physical switches. Used to install, modify, and delete flow rules on demand. Rule installation latency: <5ms for pre-installed rules, 8-15ms for reactive new-flow handling.

Traffic Monitor	Continuously reads: link utilisation per path, switch queue depths, vehicle count per fog zone, pending Super-Task sizes in the Aggregator queue. Feeds this data to both RL agents as part of their state vectors.

2.5 Layer 3 — Cloud

The cloud receives two types of workloads: boulders (Step 4 of every DAG — the GPU-heavy object classification) and Super-Tasks (Aggregator bundles during peak load). Cloud servers run 8,000 MIPS — four times faster than fog nodes per task, but with 20–50ms additional WAN propagation delay. Results are returned to the fog layer which forwards them to the vehicle via 5G.

3. Offline Optimisation — TOF-MMDE-NSGA-II

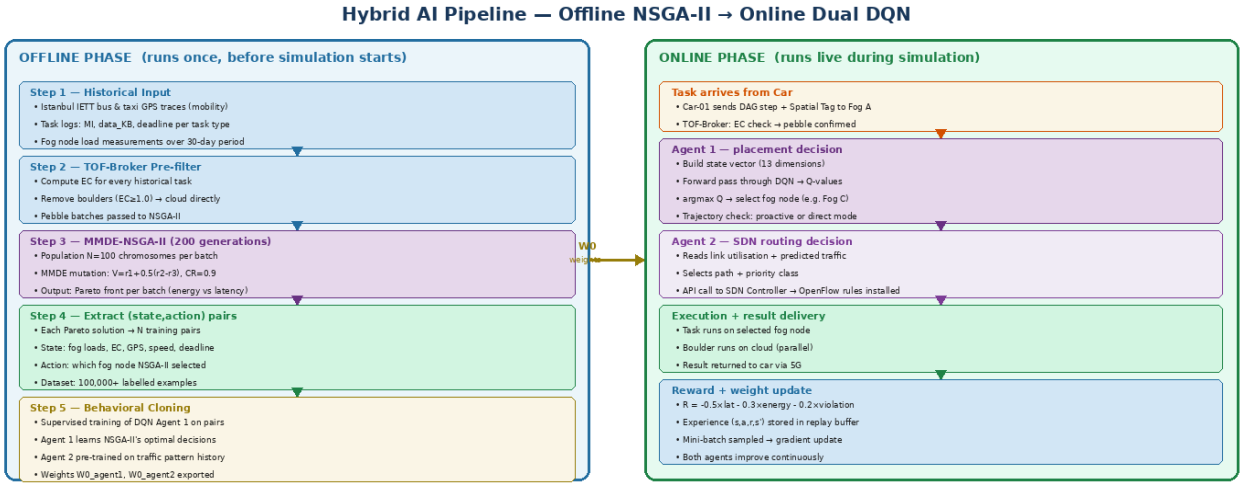

Figure 3: Hybrid AI pipeline. Left: offline phase runs TOF-MMDE-NSGA-II on historical Istanbul traffic data to generate Pareto-optimal routing plans, converted to (state,action) training pairs for behavioral cloning of both DQN agents. Right: online phase deploys both pre-trained agents live, updating weights continuously from real simulation rewards.

3.1 Why NSGA-II Runs Offline

NSGA-II runs 200 generations to find good solutions. On a batch of 100 tasks that takes several seconds of CPU time — far too slow for real-time 200ms deadlines. But the solutions it produces are mathematically verified, multi-objective optimal routing plans. The key insight: these solutions are used not for live decisions, but as training data for the DQN agents that will make live decisions instantly.

3.2 The TOF-Broker + MMDE-NSGA-II Combination

Paper 1 introduced MMDE-NSGA-II without any pre-filter. Paper 2 introduced the TOF-Broker with plain NSGA-II. This thesis combines both for the first time. The Broker reduces the search space by removing boulders before NSGA-II runs. MMDE mutation then finds better solutions faster on the reduced problem. The combination produces higher-quality Pareto fronts in fewer generations than either approach alone.

MMDE MUTATION FORMULA	Mutant V = r1 + F × (r2 - r3) where F=0.5, CR=0.9. r1, r2, r3 = three distinct solutions from population. Differential (r2-r3) provides directional guidance toward productive search regions. Smarter than random mutation — learns from population structure.

3.3 Generating Training Data

After NSGA-II converges, the knee point of the Pareto front is selected — the solution with the best balance between energy and latency. This solution is decomposed into individual (state,

action) training pairs: one pair per pebble task step. The state captures everything observable at the moment the routing decision was made. The action is the fog node NSGA-II assigned. Across all historical batches, this produces 100,000+ labelled examples.

State dimension	What it measures	Why it matters for placement
fog_cpu_A, B, C	Current CPU utilisation of each fog node (0.0–1.0)	From Traffic Monitor readings at decision time
queue_depth_A, B, C	Number of tasks queued at each fog node	Higher queue = longer effective wait time
task_EC	Execution Cost of this specific DAG step (0.0–1.0)	Determines how long the step takes per node
bandwidth_util	Current upload link utilisation (0.0–1.0)	Affects transmission time for task data
vehicle_speed	Current speed in km/h (from Spatial Tag)	Faster vehicle = smaller T_exit window
vehicle_heading	Direction in degrees (from Spatial Tag)	Used with position to project trajectory
T_exit	Seconds until vehicle leaves current fog zone	Core proactive handoff trigger variable
deadline_remaining	Milliseconds left before task deadline	Urgent tasks need fast nodes; slack allows savings
cloud_load	Current cloud processing backlog (0.0–1.0)	Affects whether cloud is a viable alternative

4. RL Agent 1 — Task Placement Brain

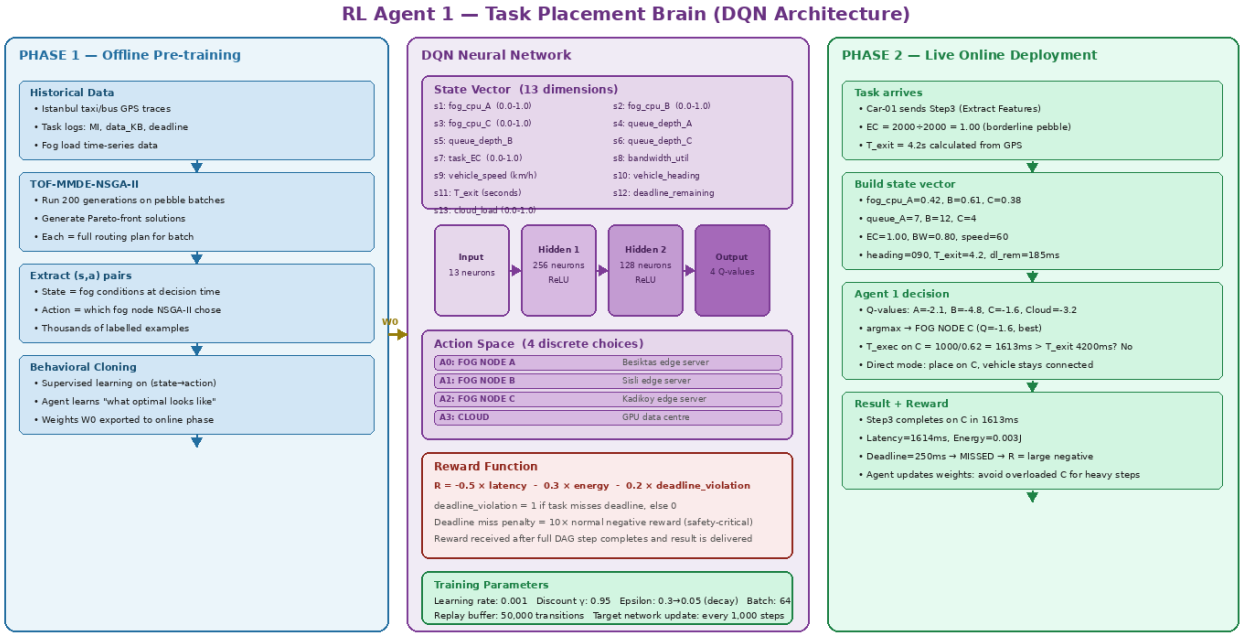

Figure 4: RL Agent 1 full architecture. Left: offline pre-training phase uses NSGA-II (state,action) pairs for behavioral cloning. Centre: DQN architecture — 13-dimensional input, two hidden layers (256→128 neurons with ReLU), 4 Q-value outputs. Bottom: reward function and training parameters. Right: live deployment example with Car-01 in Istanbul, showing state vector construction, action selection, and reward feedback.

4.1 What Agent 1 Does

RL Agent 1 is the compute placement brain. It sits in the fog layer's Application Logic subsystem and receives one request per pebble DAG step: 'where should this task step run — Fog Node A, B, C, or the Cloud?' It makes this decision in under 1ms (one neural network forward pass) and outputs the assignment. This is the decision that determines whether the task meets its deadline and how much energy it consumes.

4.2 Neural Network Architecture

Property	Detail
Network type	Deep Q-Network (DQN). Chosen because the action space is discrete and small (4 choices). DQN is well-understood, stable with experience replay, and faster to train than policy gradient methods.
Input layer	13 neurons — one per state vector dimension (see Section 3.3). Values normalised to [0,1] range.
Hidden layer 1	256 neurons, ReLU activation. Learns low-level feature combinations from state variables.

Hidden layer 2	128 neurons, ReLU activation. Learns higher-order patterns in combined features.
Output layer	4 neurons — one Q-value per action (Fog A, Fog B, Fog C, Cloud). Agent selects argmax(Q-values).
Action selection	Epsilon-greedy: probability ε of random exploration (starts 0.30, decays to 0.05 over first 10,000 steps), probability 1-ε of argmax exploitation.
Experience replay	Transitions (state, action, reward, next_state) stored in 50,000-entry buffer. Random mini-batches of 64 sampled to break correlation between consecutive experiences.
Target network	A frozen copy of the Q-network updated every 1,000 steps. Provides stable training targets and prevents divergence.

4.3 Reward Function

REWARD FORMULA	R = -0.5 × normalised_latency - 0.3 × normalised_energy - 0.2 × deadline_violation | deadline_violation = 1 if task misses deadline, 0 otherwise. Deadline miss multiplied by 10× for safety-critical steps (collision check).

The reward is received after the full DAG step completes and the result is confirmed delivered to the vehicle. The three-term structure reflects the two optimization objectives (latency, energy) plus a hard constraint (deadline). The 10× penalty on safety-critical steps ensures Agent 1 prioritises meeting deadlines for collision avoidance tasks above all else.

4.4 Training Parameters

Parameter	Value and reasoning
Learning rate	0.001 — standard Adam optimizer. Small enough for stable convergence, fast enough to adapt to changing city conditions during simulation.
Discount factor γ	0.95 — values future rewards at 95% of immediate rewards. Appropriate for tasks where the consequence of a placement decision (meeting deadline) is observed within 1–2 seconds.
Batch size	64 transitions sampled from replay buffer per update step. Balance between gradient quality and computational cost.
Update frequency	One gradient update per task completion. Continuous learning throughout simulation runtime.
Pre-training	100,000 NSGA-II (state,action) pairs used for behavioral cloning before live simulation. Eliminates cold-start. Agent enters deployment with near-optimal initial policy.

4.5 Concrete Example — Car-01 in Besiktas

Car-01 is on the E-5 highway approaching Besiktas at 70 km/h. It submits Step 3 (Feature Extraction, 2000 MI). Agent 1 builds its state vector: fog_cpu_A=0.42, fog_cpu_B=0.61,

fog_cpu_C=0.38, queue_A=7, queue_B=12, queue_C=4, EC=1.00, BW=0.80, speed=70, heading=090, T_exit=3.6s, deadline_remaining=185ms, cloud_load=0.45.

Agent 1 computes Q-values: Q(Fog A)=-2.1, Q(Fog B)=-4.8, Q(Fog C)=-1.6, Q(Cloud)=-3.2. It selects Fog C (highest Q-value). Fog C has EC=1.00/(1-0.38)=1613ms effective exec — but deadline is 200ms total and 185ms remaining. The task will miss its deadline. Agent 1 receives a large negative reward and updates its weights: 'EC=1.00 borderline pebbles on loaded nodes should escalate to Cloud faster.'

5. RL Agent 2 — SDN Network Brain

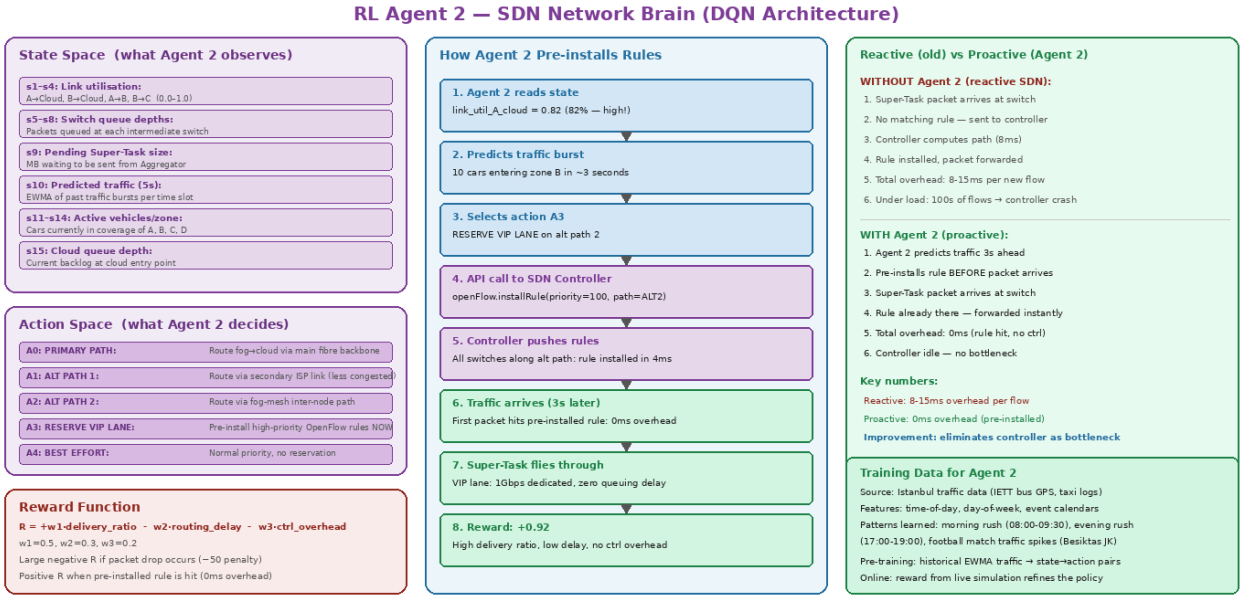

Figure 5: RL Agent 2 full architecture. Left: 15-dimensional state space (link utilisation, queue depths, pending traffic, active vehicles per zone). Centre: 5-action space and step-by-step proactive flow rule pre-installation sequence. Right: reactive vs. proactive comparison showing 8-15ms overhead eliminated by pre-installation. Training uses Istanbul traffic patterns (IETT data, event calendars, rush hours).

5.1 What Agent 2 Does

RL Agent 2 is the network routing brain. It is embedded directly inside the SDN Controller — it is not a separate service, it IS the intelligence of the controller. It observes the physical network state and decides: which path should each traffic flow take, and should it pre-install routing rules before the traffic even arrives?

The critical difference from a standard SDN controller is proactivity. A standard controller waits for a packet to arrive, notices no rule matches, sends the packet to the controller, computes a path, installs a rule, and forwards the packet. This takes 8–15ms per new flow and causes controller bottlenecks under load. Agent 2 predicts traffic 3–5 seconds ahead and installs rules before the first packet arrives — reducing per-flow overhead to zero.

5.2 State Space

State dimension	What it captures
s1–s4: Link utilisations	Current bandwidth usage on four key paths: Fog A→Cloud, Fog B→Cloud, Fog A→B, Fog B→C. Range 0.0–1.0. High values indicate impending congestion.
s5–s8: Switch queue depths	Number of packets queued at each intermediate switch. Growing queues indicate congestion building before utilisation metric responds.

s9: Pending Super-Task size	Megabytes currently queued in the Aggregator waiting to be sent. Lets Agent 2 pre-reserve bandwidth before the burst hits.
s10: Predicted traffic (5s)	Exponentially weighted moving average of traffic bursts in the past 10 time slots. Agent 2 learns to predict future load from this signal.
s11–s14: Active vehicles per zone	Number of smart cars currently in coverage of each of the four fog zones. More cars = more upcoming task traffic per zone.
s15: Cloud queue depth	Current backlog at the cloud entry point. Affects whether routing to cloud is actually fast or will face queuing delay.

5.3 Action Space and Reward

Action	Effect on network
A0: Primary path	Route current flow via main fibre backbone (lowest latency, most congested during peak).
A1: Alternative path 1	Route via secondary ISP link — less congested but slightly higher base latency.
A2: Alternative path 2	Route via fog-mesh inter-node path — useful when backbone is saturated.
A3: Reserve VIP lane	Pre-install high-priority OpenFlow rules NOW on the predicted best path. Used 3–5 seconds before anticipated traffic burst arrives.
A4: Best effort	Normal priority, no reservation — for low-priority background telemetry traffic.

REWARD FORMULA (Agent 2)	R = +0.5 × delivery_ratio - 0.3 × routing_delay - 0.2 × controller_overhead | Packet drop penalty: -50 (large, discourages any packet loss). Pre-installed rule hit bonus: +0.3 (rewards proactive behaviour directly).

5.4 Pre-training on Istanbul Traffic Data

Agent 2 is pre-trained on real Istanbul traffic patterns before the simulation begins. The training data comes from IETT public transport GPS traces (buses and trams whose routes are known), historical vehicle count data per district, and the city's event calendar (football matches at Besiktas JK's Vodafone Park stadium, for example, create predictable traffic spikes in the Besiktas zone on match days).

The agent learns: morning rush (08:00–09:30) means high Besiktas→Sisli traffic as commuters drive toward the financial district. Evening rush (17:00–19:00) reverses the flow. Match nights cause localised spikes around Besiktas zone. This temporal knowledge allows Agent 2 to pre-install VIP lanes minutes before they are actually needed — before any vehicle has even submitted a task.

6. Mobility Management — GPS Prediction and Handoff

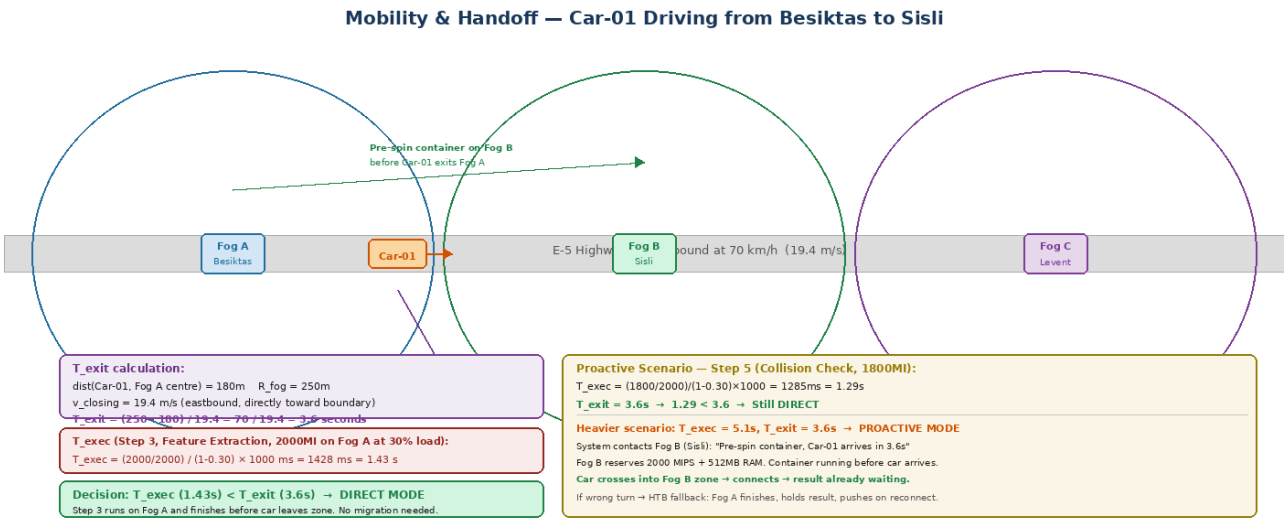

Figure 6: Mobility scenario — Car-01 driving E-5 highway from Besiktas to Sisli at 70 km/h. T_exit = 3.6 seconds until car leaves Fog A's zone. For Step 3 (T_exec=1.43s): direct mode — task finishes before car leaves. For a heavier scenario (T_exec=5.1s): proactive mode — Fog B pre-spins container before car arrives. If trajectory prediction fails: HTB reactive fallback activates.

6.1 The T_exit Formula

T_EXIT CALCULATION	T_exit = (R_fog - dist(car, fog_centre)) / v_closing | R_fog = 250m (fog coverage radius). dist() = Euclidean distance from car's GPS to fog node centre. v_closing = component of car velocity directed toward zone boundary. Example: dist=180m, v=19.4m/s → T_exit = (250-180)/19.4 = 3.6 seconds.

The Trajectory Predictor runs this calculation on every incoming task. It also estimates T_exec — how long the task will take on the current fog node accounting for node load. The comparison T_exec vs T_exit determines the operating mode.

6.2 Three Operating Modes

Mode	When it activates and what happens
Direct mode (T_exec < T_exit)	Task will finish before car leaves current zone. Normal placement by Agent 1. No migration needed. Example: Step 2 (200MI, 100ms exec) with T_exit=3.6s — finishes 3.5 seconds before zone boundary.
Proactive mode (T_exec > T_exit)	Task will NOT finish before car leaves. System identifies the next fog zone by projecting trajectory beyond T_exit. Sends a pre-spin request to that fog node: 'reserve CPU + RAM, Car-01 arrives in T_exit seconds.' Container starts immediately. Car connects to Fog B and task is already running there.

HTB fallback (prediction failed)	Car takes unexpected turn or GPS signal lost. Car disconnects from Fog A mid-task. Task is immediately moved from NTB to HTB — maximum CPU priority. Fog A completes the task as fast as possible and holds the result. Car reconnects to any fog node → Central Tracker delivers held result. Zero re-submission, zero packet loss.

6.3 Per-Step Mobility for DAG Tasks

A key innovation over existing mobility-aware systems is that this architecture applies the T_exit check individually to each DAG step. Step 2 might finish in 100ms — well within T_exit. Step 3 might take 1,600ms — requiring proactive migration. Step 5 is so fast it always finishes locally. The system makes a separate mode decision per step, not per whole task.

This matters because different steps have different MI demands and different node assignments (Agent 1 may put Step 2 on Fog A and Step 3 on Fog C). If the vehicle is moving toward Fog C's zone while Step 3 runs there, no migration is needed even if T_exit for Fog A is small — because Step 3 is already on Fog C which the vehicle is approaching.

7. Simulation Design

7.1 Environment Setup

Parameter	Value and justification
Simulation tool	iFogSim (Java) for fog/cloud computation layer + Python mobility module for vehicle movement. iFogSim is the standard tool used by Papers 1 and 2, ensuring comparability of results.
City grid	1,000m × 1,000m area representing a simplified Istanbul district. Roads at defined coordinates. Four fog nodes at (200,500), (500,200), (800,500), (500,800) — representing Besiktas, Sisli, Kadikoy, Uskudar.
Vehicle mobility	Random Waypoint Model constrained to road grid. Each vehicle selects a destination, moves at constant speed, selects new destination on arrival. Speed randomly drawn from Gaussian(μ=60, σ=15 km/h) per vehicle per trip.
Task generation	Each vehicle generates one 5-step DAG per 100ms (10fps camera). Total task rate = N_vehicles × 10 DAGs/second. Step MI values fixed as per Section 1.2.
Simulation duration	600 seconds (10 minutes) per run. Warm-up period: first 60 seconds (agents still learning — excluded from results). Results collected from seconds 60–600.

7.2 Key Parameters

Parameter	Value	Notes
Fog node MIPS	2,000 MIPS per node	Each fog node: 2B instructions/second
Cloud MIPS	8,000 MIPS	4× faster than fog, +20-50ms WAN
Fog coverage radius	250 metres	Dense urban 5G deployment
EC threshold θ	1.0 second	Boulder/pebble boundary
NSGA-II population N	100 chromosomes	Standard for this problem class
NSGA-II generations	200	Sufficient for convergence
MMDE scaling F	0.5	From Paper 1 baseline
MMDE crossover CR	0.9	From Paper 1 baseline
DQN learning rate	0.001	Adam optimiser
DQN discount γ	0.95	Near-future focused
Replay buffer	50,000 transitions	Prevents sample correlation
Epsilon decay	0.30 → 0.05	Over first 10,000 steps
Fog-cloud bandwidth	1 Gbps	Urban fibre backbone
Fog-fog bandwidth	100 Mbps	Inter-node mesh

Fog-cloud WAN delay	20–50 ms	Simulated Istanbul–cloud
5G latency	1–3 ms	Vehicle to fog node
SDN rule install	<5 ms	Pre-installed, 8-15ms reactive

7.3 Baselines for Comparison

System	What it includes
Baseline 1 — Plain NSGA-II (Paper 1)	No broker, no RL, no mobility. NSGA-II runs on all tasks including boulders. Reactive handoff only. Maximum improvement reference.
Baseline 2 — TOF-NSGA-II (Paper 2)	TOF-Broker + plain NSGA-II (no MMDE). No RL. Reactive handoff. Shows MMDE and RL contribution.
Baseline 3 — TOF-MMDE-NSGA-II only	Full offline optimizer, no online RL agents. Static NSGA-II routing for live decisions. Reactive handoff. Shows RL agents' specific contribution.
Proposed System — Full pipeline	TOF-MMDE-NSGA-II + DQN Agent 1 + DQN Agent 2 + proactive handoff + NTB/HTB fallback. The complete architecture.

7.4 Evaluation Metrics

Metric	What it measures
Average task latency (ms)	End-to-end from Step 1 start to Step 5 result delivery. Primary metric. Must beat all baselines.
Task feasibility rate (%)	Percentage of tasks completing within 200ms deadline. Must exceed 90% for system viability.
Total energy consumption (J)	Sum of transmission + computation energy across all tasks. Secondary optimization objective.
Pareto front quality (Hypervolume)	Measures quality and diversity of NSGA-II offline Pareto front. Compares MMDE vs. plain mutation.
Handoff success rate (%)	Tasks delivered without re-submission during vehicle zone transitions. Proactive vs. reactive comparison.
Fog node utilisation balance (%)	Standard deviation of CPU utilisation across fog nodes. Lower = better load balancing.
SDN pre-installation hit rate (%)	Percentage of flows hitting pre-installed Agent 2 rules vs. requiring reactive controller response.
Boulder rejection rate (%)	Percentage of DAG steps classified as boulders. Validates EC threshold choice under different load levels.

Architecture Proposal v2 — Application: Smart City Vehicular IoT, Istanbul Urban Network.